

EM-KalmanNet: Learned Expectation-Maximization for Adaptive Tracking in Partially Known, Block-Wise Time-Varying State-Space Models

Ori Cohen, Nir Shlezinger, *Senior Member, IEEE*, and Tirza Routtenberg, *Senior Member, IEEE*

Abstract—State estimation in partially known state space (SS) models is challenging when the dynamics or observation model varies across short data blocks. Classical model-based approaches, such as the expectation-maximization (EM) Kalman filter, jointly recover the latent states and the unknown model parameters, but rely on linear-Gaussian assumptions that should accurately describe the system and require numerous forward-backward passes. Consequently, their performance and computational efficiency may deteriorate under complex and non-stationary real-world conditions. On the other hand, learned Kalman smoothers are robust to model mismatch yet cannot adapt at inference to unseen model variations without labeled data. In this work, we propose *EM-KalmanNet*, an AI-aided tracking algorithm for adaptive smoothing in block-wise time-varying SS models. The method unfolds a fixed, small number of EM-like iterations into a trainable architecture: a parameter-aware RTSNet implements a learned E-step conditioned on the current model-parameter estimate, while a lightweight M-Net implements a learned M-step that updates the state-transition or the observation matrix using empirical moments, residuals, and gradient-related statistics. The two modules are shared across the unfolded iterations and are trained offline via a dedicated three-stage procedure. During deployment, the per-block parameter estimate is propagated between consecutive blocks, enabling observation-driven adaptation without labeled online data or knowledge of the noise statistics. Experiments involving linear and nonlinear models, Gaussian and non-Gaussian noise, Lorenz attractor tracking, and acoustic source localization demonstrate that EM-KalmanNet consistently outperforms model-based and data-driven benchmarks while substantially reducing inference latency relative to the EM-KF.

Index Terms—Kalman filter, deep unfolding, Expectation-Maximization (EM), Time-varying systems.

I. INTRODUCTION

Tracking the latent states of dynamical systems from noisy observations is a fundamental problem in signal processing [2], with applications ranging from navigation and localization to smart transportation and autonomous systems [3]. Classical tracking methods are effective when the statistical relations governing the state evolution and observations are accurately known. Often in practice, however, these relations are only partially characterized and may themselves evolve over time as the operating conditions, motion patterns, or sensing environment change [4]. When such variations are known, they can be directly incorporated into a time-varying state space (SS) model [5]; the

more challenging setting arises when the variations are unknown and must be inferred from the same noisy observations used for state estimation. This difficulty is further amplified when changes occur rapidly, such that only short observation intervals are available for identifying the current dynamics.

Model-based tracking is commonly built upon the Kalman filter (KF) [6], the Rauch-Tung-Striebel (RTS) smoother [7], and their nonlinear extensions [8]–[10]. When the governing model is unknown or time varying, these estimators can be augmented with mechanisms for system identification and parameter adaptation [11]. Representative approaches include multiple-model tracking for switching dynamics [12], joint or dual state and parameter estimation [13], [14], and adaptive estimation of unknown noise statistics [15]. A particularly principled approach is the expectation-maximization (EM) algorithm [16], which enables joint inference of latent states and unknown model parameters by alternating between smoothing under the current parameter estimate and updating the parameters from the inferred state statistics [17], [18]. However, such model-based adaptive methods remain tied to the assumed probabilistic description: their reliability can deteriorate under model mismatch and non-Gaussian disturbances, while standard closed-form EM-KF updates are primarily associated with linear-Gaussian SS models. Moreover, iterative parameter estimation methods such as EM require repeated forward-backward passes and typically benefit from sufficiently long observation sequences, making reliable low-latency adaptation particularly challenging when the dynamics change over short time intervals.

Data-driven state estimators provide an alternative by learning aspects of the underlying dynamics and inference rule directly from data [19]. Deep latent SS models and recurrent estimators have been developed to learn nonlinear temporal dependencies and state representations without requiring a complete analytical model [20]–[24]. A complementary line of work follows the model-based deep learning paradigm and retains the algorithmic structure of classical Bayesian estimators to leverage partial characterization of the dynamics while learning those components that are difficult to specify analytically [25]. For example, EKNet [26] differentiates through the EKF to learn unknown noise covariance parameters from data while retaining the model-based filtering structure. In particular, KalmanNet [27] replaces the analytically computed Kalman gain with a learned recurrent mapping, allowing KF-type tracking under partially known dynamics and unknown noise statistics, while RTSNet [28] extends this methodology to smoothing. Related variants further improve robustness or accommodate high-dimensional

Parts of this work were presented at the 2026 Sensor Array and Multichannel Signal Processing Workshop (SAM) as the paper [1]. O. Cohen, N. Shlezinger, and T. Routtenberg are with the School of ECE, Ben-Gurion University of the Negev, Israel (e-mail: orico@post.bgu.ac.il; {nirshl; tirzar}@bgu.ac.il). This work was supported by the Israel Science Foundation (ISF) under grant no. 3314/25 and grant no. 1148/22.

observations, and alternative learned gain architectures [29]–[31]. These approaches can substantially reduce sensitivity to inaccuracies in the assumed SS model; nonetheless, their learned inference rules are determined by the distributions represented during training, and can consequently deteriorate when the model encountered at deployment deviates from the training conditions.

Several approaches have therefore sought to endow learned Kalman-type estimators with adaptation capabilities. Unsupervised KalmanNet [32] exploits observation-prediction errors to adapt a pre-trained KalmanNet to changing SS models without requiring ground-truth state labels. However, it requires updating the learned model through lengthy gradient-based optimization at deployment. While such gradient-based learning can be made more efficient by drift-aware learning rate adaptation [33] or by incorporating model-agnostic meta-learning techniques [34], such a training method may still be unsuitable for coping with rapid variations. Adaptive KalmanNet [35] uses a compact hypernetwork to modulate the learned filtering rule according to context information describing the current operating regime, enabling adaptation without retraining, assuming that the variations are encompassed by the offline training data, and that such contextual information is available. Still, there remains a gap between classical adaptive estimators, which can infer changing model parameters directly from observations but rely strongly on prescribed statistical models, and learned state estimators, which are robust to model mismatch but lack an efficient mechanism for jointly inferring the latent states and unknown time-varying model parameters from unlabeled observations.

Motivated by this gap, we propose *EM-KalmanNet*, a model-based deep learning methodology for adaptive smoothing in partially known, block-wise time-varying SS models. EM-KalmanNet combines the observation-driven parameter adaptation of the EM-KF with the robustness of KalmanNet-type learned estimators by unfolding a fixed, small number of EM-like iterations into a discriminative trainable architecture [36]. Rather than retaining the analytical EM updates, we learn both stages of the iterative procedure: a parameter-aware RTSNet [28] implements the smoothing step conditioned on the current model-parameter estimate, while a lightweight module termed *M-Net* learns to refine this estimate from model-inspired statistics of the smoothed trajectory. The resulting architecture is trained entirely offline and, at deployment, jointly refines the latent states and the unknown model parameters directly from the observations, without labeled online data, online network optimization, or knowledge of the noise statistics.

Our main contributions are summarized as follows:

- We introduce an EM-inspired neural architecture for adaptive smoothing in partially known and time-varying SS models. The proposed EM-KalmanNet unfolds a prescribed number of EM-like iterations and replaces their analytical components with shared trainable modules, yielding a fixed-complexity inference procedure that combines model-based structure with data-driven robustness. We consider both unknown time-varying state-transition and observation models, while allowing the available part of the SS model to be nonlinear and the noise statistics to be unknown.
- We design dedicated learned E- and M-steps for joint state and parameter inference. The E-step is implemented by

a parameter-aware RTSNet whose model-based recursions and learned gains are conditioned on the current parameter estimate. The M-step is realized by an M-Net that directly learns a correction to the unknown model parameter from compact empirical moments, residual statistics, and gradient-related quantities motivated by the analytical EM update. We further show that a covariance-agnostic counterpart of the classical EM M-step can be expressed within this correction-based formulation, while the learned M-Net is free to realize more general updates suitable for model mismatch and non-Gaussian settings.

- We develop a dedicated multi-stage training procedure for the unfolded architecture. The parameter-aware RTSNet is first trained to smooth under both accurate and perturbed parameter values, after which the M-Net is trained to perform parameter refinement with the smoother fixed, followed by joint fine-tuning of the complete unfolded estimator. Sharing the learned modules across iterations and training them to operate on imperfect incoming parameter estimates enables observation-driven adaptation with a small prescribed inference budget and permits the parameter estimate to be propagated between consecutive blocks.
- We evaluate EM-KalmanNet across a broad set of time-varying tracking scenarios involving unknown state-transition and observation models, nonlinear dynamics and observations, Gaussian and non-Gaussian disturbances, short observation blocks, Lorenz-system tracking, and acoustic source localization. The results demonstrate robust adaptation under model mismatch and changing dynamics, with improved state-estimation performance over mismatched model-based and learned benchmarks across the considered settings, while requiring substantially lower inference latency than iterative EM-KF processing.

The rest of this paper is organized as follows: Section II formulates the problem. EM-KalmanNet is detailed in Section III, and evaluated in Section IV. Section V provides concluding remarks.

Notation: Vectors and matrices are denoted by bold lowercase and uppercase letters, respectively. The operators $(\cdot)^\top$, $\text{vec}(\cdot)$, $\mathbb{E}[\cdot]$, and $\text{Cov}(\cdot)$ denote transpose, vectorization, expectation, and covariance, respectively. Moreover, $\|\cdot\|_2$ and $\|\cdot\|_F$ denote the Euclidean and Frobenius norms, and $\mathbf{I}_m$ is the $m \times m$ identity matrix. For a sequence $\{\mathbf{a}_t\}_{t=1}^T$, we use $\mathbf{a}_{1:T} \triangleq (\mathbf{a}_1, \dots, \mathbf{a}_T)$ to denote the corresponding length- T sequence.

II. SYSTEM MODEL AND PRELIMINARIES

In this section, we present the system model and the required background for the proposed architecture. In Subsection II-A, we formulate the block-wise time-varying SS model and present the main challenges. In Subsection II-B, we review the main building blocks underlying EM-KalmanNet, including Kalman smoothing, EM-KF procedures, and RTSNet, thereby setting the basis for the EM-KalmanNet architecture developed in Section III.

A. System Model

Signal Model: We consider a discrete-time dynamic system with hidden state $\mathbf{x}_t \in \mathbb{R}^m$ that is related to an observation

$\mathbf{y}_t \in \mathbb{R}^n$ according to a SS model of the form

$$\mathbf{x}_t = f_t(\mathbf{x}_{t-1}) + \mathbf{w}_t, \quad \mathbf{y}_t = h_t(\mathbf{x}_t) + \mathbf{v}_t. \quad (1)$$

In (1), $f_t(\cdot)$ is the state evolution function, and $h_t(\cdot)$ is the observation function. The process and observation noises, $\mathbf{w}_t$ and $\mathbf{v}_t$, respectively, are assumed to be zero-mean, mutually independent, temporally independent, and independent of the initial state, $\mathbf{x}_0$. We focus on settings in which the underlying SS model varies over time in a block-wise manner, such that the state-transition and observation functions, $f_t(\cdot)$ and $h_t(\cdot)$, remain fixed within each block and may vary between consecutive blocks. Each block consists of T time instants. For a block containing times $\tau + 1, \dots, \tau + T$, $\mathbf{x}_\tau$ serves as its initial state for the corresponding SS model.

Problem Formulation: We consider a block-wise smoothing problem in which, at block index i , $\mathbf{X}_i = \{\mathbf{x}_\tau\}_{\tau=(i-1)T+1}^{iT}$ is estimated from the observed sequence $\mathbf{Y}_i = \{\mathbf{y}_\tau\}_{\tau=(i-1)T+1}^{iT}$. Each trajectory consists of M consecutive blocks, where the SS model is fixed within each block and may vary between blocks. Given $\mathbf{Y}_i$, the prior first- and second-order moments of the initial state $\mathbf{x}_{(i-1)T}$ obtained from the previous block, and an initial estimate of the unknown parameter, the goal is to estimate $\mathbf{X}_i$ while adapting the unknown parameter.

We particularly focus on smoothing subject to the following challenges:

- C.1** The functions $f_t(\cdot)$ and $h_t(\cdot)$ may be crude approximations of the true underlying model.
- C.2** The distribution of the noise signals is unknown and may be non-Gaussian.
- C.3** The variations in $f_t(\cdot)$ and $h_t(\cdot)$ are unknown.
- C.4** The estimate of $\mathbf{X}_i$ must be produced with minimal latency.
- C.5** Variations in $f_t(\cdot)$ or $h_t(\cdot)$ can occur rapidly, requiring operation with a small block length T , which leaves few observations for reliable smoothing and parameter adaptation.

To cope with **C.1-C.5**, we assume that during training, one has access to data from the underlying SS model. Such data are given by

$$\mathcal{D} = \left\{ \left\{ \{(\mathbf{x}_{j,i,t}, \mathbf{y}_{j,i,t})\}_{t=1}^T, f_{j,i}, h_{j,i} \right\}_{i=1}^M \right\}_{j=1}^{N_{\mathcal{D}}}. \quad (2)$$

Here, $N_{\mathcal{D}}$ denotes the number of training trajectories, while $f_{j,i}$ and $h_{j,i}$ denote the nominal state-transition and observation models available to the estimator during training. Such side information naturally arises in model-based signal processing systems from first-principles modeling, system identification, or calibration, and may itself be approximate, as also demonstrated in the experimental study.

Cases Considered: Throughout the paper, we focus on two complementary scenarios, distinguished by which of the two functions in (1) is unknown. In both cases, the available function is used as is, while the unknown function is estimated from data and is parameterized as linear within each block.

Case 1 (Unknown State Transition Function): The state-transition function $f_t(\cdot)$ is unknown and varies between blocks; it is parameterized by a transition matrix $\mathbf{F}$, while $h_t(\cdot)$ is available. In this case, (1) is represented, within a given block, by the model

$$\mathbf{x}_t = \mathbf{F}\mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{y}_t = h_t(\mathbf{x}_t) + \mathbf{v}_t. \quad (3)$$

Case 2 (Unknown Observation Function): The observation function $h_t(\cdot)$ is unknown and varies between blocks; it is parameterized by an observation matrix $\mathbf{H}$, whereas the state-transition function $f_t(\cdot)$ is available. In this case, (1) is represented, within a given block, by the model

$$\mathbf{x}_t = f_t(\mathbf{x}_{t-1}) + \mathbf{w}_t, \quad \mathbf{y}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t. \quad (4)$$

B. Preliminaries

Kalman Smoothing: The classical approach for smoothing is based on variants of the KF, such as the extended RTS algorithm [7]. For linear Gaussian SS models with known parameters, the KF is optimal for online filtering, while the RTS smoother is optimal for block smoothing in the minimum mean squared error (MMSE) sense. Such methods are designed for dynamics that are captured by a known SS model (violating **C.1** and **C.3**) with known Gaussian state and observation noise covariances $\mathbf{Q}$ and $\mathbf{R}$, respectively (violating **C.2**).

To describe the extended RTS smoother, we focus on a single block of length T , indexed locally as $t = 1, \dots, T$. The initial prior information is represented by $\mathbf{x}_0$, from which the state trajectory evolves according to the SS model. The smoother applies a forward pass followed by a backward pass to estimate the latent state trajectory $\mathbf{x}_{1:T}$ within the block. The forward pass is an extended KF (EKF), which updates its *prior* state and covariance estimates based on past observations. For each t , the forward pass computes the predicted first-order moments via

$$\hat{\mathbf{x}}_{t|t-1} = f_t(\hat{\mathbf{x}}_{t-1|t-1}), \quad \hat{\mathbf{y}}_{t|t-1} = h_t(\hat{\mathbf{x}}_{t|t-1}), \quad (5)$$

and the predicted second-order moments via

$$\hat{\mathbf{P}}_{t|t-1} = \hat{\mathbf{F}}_t \hat{\mathbf{P}}_{t-1|t-1} \hat{\mathbf{F}}_t^\top + \mathbf{Q}, \quad \hat{\mathbf{S}}_t = \hat{\mathbf{H}}_t \hat{\mathbf{P}}_{t|t-1} \hat{\mathbf{H}}_t^\top + \mathbf{R},$$

where $\hat{\mathbf{F}}_t = \left. \frac{\partial f_t(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1|t-1}}$, and $\hat{\mathbf{H}}_t = \left. \frac{\partial h_t(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_{t|t-1}}$ denote the local linearizations of $f_t(\cdot)$ and $h_t(\cdot)$. The predictions are updated via

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathcal{K}_t(\mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}), \quad \hat{\mathbf{P}}_{t|t} = \hat{\mathbf{P}}_{t|t-1} - \mathcal{K}_t \hat{\mathbf{S}}_t \mathcal{K}_t^\top, \quad (6)$$

with *forward* Kalman gain (KG) $\mathcal{K}_t = \hat{\mathbf{P}}_{t|t-1} \hat{\mathbf{H}}_t^\top \hat{\mathbf{S}}_t^{-1}$.

The backward pass refines the forward estimates by incorporating information from future observations, applied sequentially from $t = T - 1$ to $t = 1$. For each t , the state is updated via

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_{t|t} + \mathcal{G}_t(\hat{\mathbf{x}}_{t+1} - \hat{\mathbf{x}}_{t+1|t}), \quad (7)$$

while the corresponding covariance is updated via

$$\hat{\mathbf{P}}_t = \hat{\mathbf{P}}_{t|t} + \mathcal{G}_t(\hat{\mathbf{P}}_{t+1} - \hat{\mathbf{P}}_{t+1|t}) \mathcal{G}_t^\top. \quad (8)$$

Here, the backward KG is $\mathcal{G}_t = \hat{\mathbf{P}}_{t|t} \hat{\mathbf{F}}_{t+1}^\top \hat{\mathbf{P}}_{t+1|t}^{-1}$.

EM-KF: The classical treatment of smoothing with unknown parameters considers a linear Gaussian instance of (1), i.e., $f_t(\mathbf{x}) = \mathbf{F}\mathbf{x}$, $h_t(\mathbf{x}) = \mathbf{H}\mathbf{x}$, and Gaussian $\mathbf{w}_t$ and $\mathbf{v}_t$ with known covariances $\mathbf{Q}$, $\mathbf{R}$. In this setting, the EM algorithm can estimate the unknown parameters and the latent states [18].

The EM algorithm iteratively forms the expected complete-data log-likelihood of the observations,

$$\mathcal{Q}(\phi; \phi^{(l-1)}) = \mathbb{E}_{\mathbf{x}_{1:T} | \mathbf{y}_{1:T}; \phi^{(l-1)}} [\log p(\mathbf{x}_{1:T}, \mathbf{y}_{1:T}; \phi)], \quad (9)$$

and then selects the parameter that maximizes this quantity. In (9), the unknown parameter is ϕ , and $\hat{\phi}^{(l)}$ is its estimate after l EM iterations. The method starts with an initial estimate $\phi^{(0)}$, and alternates between two steps:

(i) The *E-step* applies the RTS smoother under the current parameter estimate to compute the posterior moments of the latent state trajectory required for the expected complete-data log-likelihood. At iteration l , it assumes a linear Gaussian SS model with the current parameter estimate $\phi^{(l-1)}$, and computes

$$\left\{ \hat{\mathbf{x}}_t^{(l)}, \hat{\mathbf{P}}_t^{(l)}, \hat{\mathbf{P}}_{t,t-1}^{(l)} \right\}_{t=1}^T,$$

where the lag-one smoothed cross-covariance is given by

$$\hat{\mathbf{P}}_{t,t-1}^{(l)} = \text{Cov}(\mathbf{x}_t, \mathbf{x}_{t-1} | \mathbf{y}_{1:T}; \phi^{(l-1)}), \quad (10)$$

computed using the standard RTS smoother recursion.

(ii) The *M-step* updates the unknown parameter by maximizing the expected log-likelihood using the moments computed in the E-step. To simplify the M-step notation, we define the following EM sufficient statistics:

$$\mathbf{A}_1^{(l)} \triangleq \sum_{t=1}^T \left(\hat{\mathbf{x}}_t^{(l)} (\hat{\mathbf{x}}_{t-1}^{(l)})^\top + \hat{\mathbf{P}}_{t,t-1}^{(l)} \right), \quad (11a)$$

$$\mathbf{A}_2^{(l)} \triangleq \sum_{t=1}^T \left(\hat{\mathbf{x}}_{t-1}^{(l)} (\hat{\mathbf{x}}_{t-1}^{(l)})^\top + \hat{\mathbf{P}}_{t-1}^{(l)} \right), \quad (11b)$$

$$\mathbf{A}_3^{(l)} \triangleq \sum_{t=1}^T \left(\hat{\mathbf{x}}_t^{(l)} (\hat{\mathbf{x}}_t^{(l)})^\top + \hat{\mathbf{P}}_t^{(l)} \right), \quad (11c)$$

$$\mathbf{A}_4^{(l)} \triangleq \sum_{t=1}^T \mathbf{y}_t (\hat{\mathbf{x}}_t^{(l)})^\top. \quad (11d)$$

The sufficient statistics in (11) also depend on the initial smoothed moments $\hat{\mathbf{x}}_0^{(l)}, \hat{\mathbf{P}}_0^{(l)}, \hat{\mathbf{P}}_{1,0}^{(l)}$, which are obtained from the initialization of the RTS smoother.

When the state-transition matrix $\mathbf{F}$ is unknown, the M-step yields

$$\mathbf{F}^{(l)} = \mathbf{A}_1^{(l)} \left(\mathbf{A}_2^{(l)} + \epsilon \cdot \mathbf{I}_m \right)^{-1}, \quad (12)$$

where $\epsilon > 0$ is a small regularization constant. When the observation matrix $\mathbf{H}$ is unknown, the M-step yields

$$\mathbf{H}^{(l)} = \mathbf{A}_4^{(l)} \left(\mathbf{A}_3^{(l)} + \epsilon \cdot \mathbf{I}_m \right)^{-1}. \quad (13)$$

In the linear Gaussian setting where the involved matrices are non-singular, this procedure monotonically increases the likelihood (for $\epsilon = 0$) until convergence [37], but typically requires multiple forward-backward passes (violating C.4).

RTSNet: The *RTSNet* algorithm proposed in [28] is a neural-augmented RTS smoother based on the KalmanNet methodology [27]. It replaces the computation of the forward and backward gains, $\mathcal{K}_t$ and $\mathcal{G}_t$ in (6)-(7), with dedicated recurrent neural networks (RNNs) trained jointly in an end-to-end manner, without explicitly tracking the second-order moments. The forward pass follows the EKF and the backward pass applies the update of (7), while the learned gain computations enable coping with approximated SS models (Challenge C.1) and non-Gaussian dynamics (Challenge C.2). However, as the learned

gains are tied to a fixed training distribution, RTSNet cannot readily accommodate time-varying SS models (Challenge C.3).

III. EM-KALMANNET

This section introduces the EM-KalmanNet algorithm, illustrated in Fig. 1. The design of EM-KalmanNet is motivated by combining the complementary strengths of the model-based EM-KF and of AI-aided KalmanNet-type smoothers to jointly address Challenges C.1–C.5. The EM-KF framework provides a principled mechanism for estimating unknown model parameters alongside the latent states, making it suited to address the unknown block-wise variations in C.3. However, it relies on linear-Gaussian assumptions, typically requires many iterations until convergence, and benefits from long observation blocks, rendering it unsuitable for Challenges C.1–C.2, C.4, and C.5. RTSNet inherits from KalmanNet the ability to operate under mismatched models and non-Gaussian noise (Challenges C.1–C.2), but cannot readily adapt to varying dynamics at inference without labeled online data.

Rather than learning an entirely new tracking architecture, EM-KalmanNet unfolds the EM iterations into a trainable architecture, replacing the analytical E- and M-step computations with learnable modules while preserving the EM-inspired estimation flow. The learned E-step is implemented by the parameter-aware RTSNet, with trainable parameters θ_E , while the learned M-step is implemented by M-Net, with trainable parameters θ_M . The separation into two modules preserves the interpretable structure of the EM-KF, allows each module to specialize in its respective task, and enables the fixed, small iteration budget to satisfy the latency requirement in Challenge C.4. We next detail the architecture in Subsection III-A and its training in Subsection III-B, and conclude with a discussion in Subsection III-C.

A. EM-KalmanNet Algorithm

We next detail the construction of EM-KalmanNet. For simplicity, we omit the block index i and describe the processing of a single block of T observations.

Initialization: EM-KalmanNet operates with an initial estimate $\hat{\phi}^{(0)}$ of the unknown model parameter. When processing consecutive blocks, $\hat{\phi}^{(0)}$ is set to the parameter estimate obtained for the previous block, exploiting the gradual block-wise variation of the model. For the first block, $\hat{\phi}^{(0)}$ can be obtained from prior model knowledge or from an offline estimate computed from the training data.

Parameter-Aware RTSNet (Learned E-Step): A key requirement of EM-KalmanNet is that the smoother used in the E-step operates reliably under different values of the unknown parameter, since the parameter estimate is refined across the unfolded iterations. Standard RTSNet is trained for a fixed SS model, and its learned gains implicitly encode that specific model; it therefore cannot directly account for variations in $\mathbf{F}$ or $\mathbf{H}$. We thus employ a *parameter-aware* RTSNet, denoted RTSNet_{PA}, in which the current parameter estimate $\hat{\phi}^{(l-1)}$ is provided as an additional feature to the RNNs that compute the forward and backward gains.

M-Net (Learned M-Step): The M-step of EM-KalmanNet is implemented by a neural module termed *M-Net*, which learns to

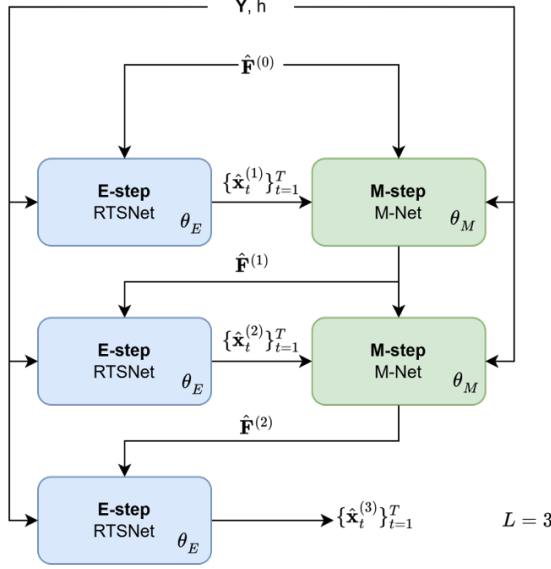


Fig. 1: EM-KalmanNet with $\phi = F$ and $L = 3$ iterations.

update the unknown model parameter from statistics computed using the smoothed trajectory. Rather than directly outputting the next parameter estimate, M-Net predicts a correction to the current estimate,

$$\Delta\phi^{(l)} = \text{MNet}(z_\phi^{(l)}; \theta_M), \quad \hat{\phi}^{(l)} = \hat{\phi}^{(l-1)} + \Delta\phi^{(l)}, \quad (14)$$

where $z_\phi^{(l)}$ is a feature vector (whose structure is detailed below), and θ_M denotes the trainable parameters of M-Net. In our numerical study in Section IV, we implemented M-Net as a fully-connected deep neural network (DNN) with GELU activations and a scaled hyperbolic tangent output layer to produce a bounded correction $\Delta\phi^{(l)}$. The correction-based update (14) is motivated by the block-wise setting, where the model varies gradually across consecutive blocks. A bounded local correction around the current estimate is easier to learn reliably than a full re-estimation, particularly under short block lengths (C.5).

Features Design: The input $z_\phi^{(l)}$ comprises two types of quantities: (i) empirical counterparts of the EM-KF sufficient statistics (11), which describe the correlation structure needed for parameter estimation; and (ii) residual- and gradient-based statistics, computed after applying the current parameter estimate, which indicate how this estimate should be corrected. Since RTSNet does not explicitly track the posterior covariance matrices $\hat{P}_t^{(l)}$ and $\hat{P}_{t,t-1}^{(l)}$, the covariance-dependent statistics of (11) are replaced with empirical moments of the smoothed trajectory. This statistics-based design avoids feeding the entire trajectory into the M-step network, yielding a compact, block-length-independent update rule that preserves the structure of the classical EM.

We next formulate how $z_\phi^{(l)}$ is designed for each of the considered cases. For brevity, we define the T -length empirical cross-covariance of time sequences $\{a_t\}$ and $\{b_t\}$ as

$$\text{Cor}_T\{a_t, b_t\} \triangleq \frac{1}{T} \sum_{t=1}^T \left(a_t - \frac{1}{T} \sum_{\tau=1}^T a_\tau \right) \left(b_t - \frac{1}{T} \sum_{\tau=1}^T b_\tau \right)^\top.$$

Case 1 (Unknown-F): For this case, we define the one-step prediction under the current estimate and its residual as

$$x_t^{-(l)} \triangleq \hat{F}^{(l-1)} \hat{x}_{t-1}^{(l)}, \quad \Delta x_t^{(l)} \triangleq \hat{x}_t^{(l)} - x_t^{-(l)}, \quad (15)$$

and the observation residual under the available observation function as $\nu_{F,t}^{(l)} \triangleq y_t - h_t(\hat{x}_t^{(l)})$. We form the transition- and observation-residual covariances $S_{\Delta x}^{(l)} \triangleq \text{Cor}_T\{\Delta x_t^{(l)}, \Delta x_t^{(l)}\}$ and $S_{\nu,F}^{(l)} \triangleq \text{Cor}_T\{\nu_{F,t}^{(l)}, \nu_{F,t}^{(l)}\}$. The matrix $S_{\Delta x}^{(l)}$ measures the empirical covariance of the transition errors induced by the current estimate $\hat{F}^{(l-1)}$: when this estimate is accurate, these errors mainly reflect the process noise, whereas an inaccurate transition matrix inflates them with a structured mismatch component. Similarly, $S_{\nu,F}^{(l)}$ quantifies the remaining error in the measurement domain.

Next, we use the cross-statistic $C_{\Delta x,x}^{(l)} \triangleq \frac{1}{T} \sum_{t=1}^T \Delta x_t^{(l)} (\hat{x}_{t-1}^{(l)})^\top$, which provides directional information for correcting F , as

$$C_{\Delta x,x}^{(l)} = -\nabla_F \left(\frac{1}{2T} \sum_{t=1}^T \left\| \hat{x}_t^{(l)} - F \hat{x}_{t-1}^{(l)} \right\|^2 \right) \Big|_{F=\hat{F}^{(l-1)}}.$$

The resulting M-Net input for the unknown- F case is

$$z_F^{(l)} = \left[(\text{vec}(\tilde{A}_1^{(l)}))^\top, (\text{vec}(\tilde{A}_2^{(l)}))^\top, (\text{vec}(S_{\Delta x}^{(l)}))^\top, (\text{vec}(S_{\nu,F}^{(l)}))^\top, (\text{vec}(C_{\Delta x,x}^{(l)}))^\top, (\text{vec}(\hat{F}^{(l-1)}))^\top \right]^\top, \quad (16)$$

where

$$\tilde{A}_1^{(l)} \triangleq \sum_{t=1}^T \hat{x}_t^{(l)} (\hat{x}_{t-1}^{(l)})^\top, \quad \tilde{A}_2^{(l)} \triangleq \sum_{t=1}^T \hat{x}_{t-1}^{(l)} (\hat{x}_{t-1}^{(l)})^\top. \quad (17)$$

Case 2 (Unknown-H): Analogously, for this case, the observation residual under the current estimate is

$$\nu_{H,t}^{(l)} \triangleq y_t - \hat{H}^{(l-1)} \hat{x}_t^{(l)} \quad (18)$$

from which we form the residual covariance $S_{\nu,H}^{(l)} \triangleq \text{Cor}_T\{\nu_{H,t}^{(l)}, \nu_{H,t}^{(l)}\}$ and residual-state cross-statistic $C_{\nu,x}^{(l)} \triangleq \frac{1}{T} \sum_{t=1}^T \nu_{H,t}^{(l)} (\hat{x}_t^{(l)})^\top$. As in Case 1, $S_{\nu,H}^{(l)}$ captures the covariance structure of the observation residuals induced by the current estimate $\hat{H}^{(l-1)}$, and therefore provides an empirical description of the remaining measurement-domain error. The cross-statistic $C_{\nu,x}^{(l)}$ is the negative gradient of the observation fitting loss $\frac{1}{2T} \sum_{t=1}^T \left\| y_t - H \hat{x}_t^{(l)} \right\|^2$ evaluated at $\hat{H}^{(l-1)}$, and thus encodes the steepest-descent correction direction.

The M-Net input for the unknown- H case is

$$z_H^{(l)} = \left[(\text{vec}(\tilde{A}_4^{(l)}))^\top, (\text{vec}(\tilde{A}_3^{(l)}))^\top, (\text{vec}(S_{\nu,H}^{(l)}))^\top, (\text{vec}(C_{\nu,x}^{(l)}))^\top, (\text{vec}(\hat{H}^{(l-1)}))^\top \right]^\top, \quad (19)$$

where

$$\tilde{A}_3^{(l)} \triangleq \sum_{t=1}^T \hat{x}_t^{(l)} (\hat{x}_t^{(l)})^\top, \quad (20)$$

Remark 1 (Relation to the analytical EM update). *The M-Net features in (16) and (19) are chosen such that the covariance-agnostic counterpart of the classical M-step can be expressed as a correction of the form in (14). Consider the*

unknown- $\mathbf{H}$ case. By the definition of $\nu_{H,t}^{(l)}$,

$$\mathbf{A}_4^{(l)} = T\mathbf{C}_{\nu x}^{(l)} + \hat{\mathbf{H}}^{(l-1)} \tilde{\mathbf{A}}_3^{(l)}. \quad (21)$$

Substituting this into the update (13), with $\mathbf{A}_3^{(l)}$ replaced by $\tilde{\mathbf{A}}_3^{(l)}$, yields

$$\hat{\mathbf{H}}^{(l)} = \hat{\mathbf{H}}^{(l-1)} + (T\mathbf{C}_{\nu x}^{(l)} - \epsilon \hat{\mathbf{H}}^{(l-1)}) (\tilde{\mathbf{A}}_3^{(l)} + \epsilon \mathbf{I}_m)^{-1}. \quad (22)$$

Thus, the update is a preconditioned gradient-type correction whose ingredients, $\mathbf{C}_{\nu x}^{(l)}$, $\tilde{\mathbf{A}}_3^{(l)}$, and $\hat{\mathbf{H}}^{(l-1)}$, are all contained in $\mathbf{z}_H^{(l)}$. An analogous identity holds for the unknown- $\mathbf{F}$ case, using $\tilde{\mathbf{A}}_1^{(l)} = T\mathbf{C}_{\Delta x, x}^{(l)} + \hat{\mathbf{F}}^{(l-1)} \tilde{\mathbf{A}}_2^{(l)}$ together with (12). The only approximation involved is the replacement of the exact EM statistics with their empirical counterparts, which omits the posterior covariance terms $\hat{\mathbf{P}}_t^{(l)}$, $\hat{\mathbf{P}}_{t,t-1}^{(l)}$ that RTSNet does not explicitly track. Consequently, M-Net can in principle reproduce this covariance-agnostic analytical EM update as a special case of (14), while its learned nonlinear mapping is free to realize more general corrections, adapted to non-Gaussian noise and model mismatch.

Overall Algorithm: EM-KalmanNet is obtained by unfolding the EM-KF flow into L EM-like iterations, where L is a small integer set to satisfy the latency requirement of Challenge C.4 (e.g., $L = 3$ as used in Section IV). In the first $L - 1$ iterations, the E-step is implemented by the shared RTSNet and is followed by an M-step implemented by the shared M-Net. In the final iteration, only the E-step is performed, so that EM-KalmanNet uses L RTSNet passes and $L - 1$ M-Net updates. The per-block processing is summarized in Algorithm 1. When operating over multiple blocks, as formulated in Subsection II-A, the initialization $\hat{\phi}^{(0)}$ of the i th block is set to the estimate $\hat{\phi}^{(L-1)}$ produced for block $i - 1$. The initial state estimate is propagated from the previous block, whereas its covariance is reinitialized at the beginning of each block.

Algorithm 1: EM-KalmanNet at one block

Init : Trained RTSNet (θ_E); trained M-Net (θ_M)

Input : Observation $\{\mathbf{y}_t\}_{t=1}^T$; initial estimate $\hat{\phi}^{(0)}$; initial state moments; available model relation $h(\cdot)$ (unknown- $\mathbf{F}$ case) or $f(\cdot)$ (unknown- $\mathbf{H}$ case);

1 **for** $l = 1, \dots, L - 1$ **do**

E-Step:

2 Set $\{\hat{\mathbf{x}}_t^{(l)}\}_{t=1}^T \leftarrow \text{RTSNet}_{\text{PA}}(\{\mathbf{y}_t\}_{t=1}^T, \hat{\phi}^{(l-1)}; \theta_E)$;

M-Step:

3 Set $\mathbf{z}_\phi^{(l)}$ via (16) or (19) ;

4 Obtain correction $\Delta\phi^{(l)} \leftarrow \text{MNet}(\mathbf{z}_\phi^{(l)}; \theta_M)$;

5 Update parameters $\hat{\phi}^{(l)} \leftarrow \hat{\phi}^{(l-1)} + \Delta\phi^{(l)}$;

Final E-Step:

6 Set $\{\hat{\mathbf{x}}_t^{(L)}\}_{t=1}^T \leftarrow \text{RTSNet}_{\text{PA}}(\{\mathbf{y}_t\}_{t=1}^T, \hat{\phi}^{(L-1)}; \theta_E)$;

7 **return** $\{\hat{\mathbf{x}}_t^{(L)}\}_{t=1}^T, \hat{\phi}^{(L-1)}$

B. Training

Algorithm 1 requires setting the parameters of the shared RTSNet, θ_E , and of the shared M-Net, θ_M . These parameters are learned offline from the dataset $\mathcal{D}$ in (2) via a three-stage training procedure, summarized in Algorithm 2. The staged design first establishes a reliable smoother, then a reliable parameter-update rule on top of it, and finally adapts both

modules to operate jointly through the unfolded iterations; this ordering avoids the optimization instabilities that arise when both modules are trained jointly from scratch.

Stage 1: Pre-Train RTSNet. We first train the parameter-aware RTSNet to recover the latent state trajectory from the observations given a model-parameter value provided as input. For each training block, indexed by (j, i) , let $\phi_{j,i}$ denote the parameter value provided to RTSNet for conditioning. Given the observations $\{\mathbf{y}_{j,i,t}\}_{t=1}^T$ and $\phi_{j,i}$, RTSNet is trained with the state-estimation loss

$$\mathcal{L}_{\mathcal{D}}^{\text{RTSNet}}(\theta_E) = \frac{1}{N_{\mathcal{D}}MT} \sum_{j=1}^{N_{\mathcal{D}}} \sum_{i=1}^M \sum_{t=1}^T \|\hat{\mathbf{x}}_{j,i,t} - \mathbf{x}_{j,i,t}\|_2^2, \quad (23)$$

where $\hat{\mathbf{x}}_{j,i,t}$ is the smoothed state produced by $\text{RTSNet}_{\text{PA}}(\{\mathbf{y}_{j,i,t}\}_{t=1}^T, \phi_{j,i}; \theta_E)$, and θ_E are the current network parameters.

To make the smoother robust to the imperfect parameter estimates encountered during the unfolded iterations, Stage 1 comprises two phases that differ in the conditioning parameter $\phi_{j,i}$: in the first phase we set $\phi_{j,i} = \phi_{j,i}^*$, i.e., the ground-truth parameter, so that RTSNet learns accurate smoothing under the correct model; in the second phase we set $\phi_{j,i} = \phi_{j,i}^* + \delta\phi_{j,i}$, where $\delta\phi_{j,i}$ is a random perturbation whose magnitude is matched to the errors anticipated at inference. The second phase deliberately exposes RTSNet to mismatched inputs, teaching it to remain informative when conditioned on inexact parameter estimates.

Stage 2: Train M-Net with Frozen RTSNet. Given the pre-trained RTSNet, we train the M-Net to update the unknown parameter while keeping the RTSNet parameters θ_E fixed. At each unfolded iteration, RTSNet produces a smoothed trajectory, from which the feature vector $\mathbf{z}_\phi^{(l)}$ is computed as described above. The M-Net then predicts an additive correction $\Delta\phi^{(l)}$. The Stage-2 loss combines a parameter-estimation error, a state-estimation term, and an update regularizer

$$\begin{aligned} \mathcal{L}_{\mathcal{D}}^{\text{M}}(\theta_M) &= \frac{1}{N_{\mathcal{D}}M} \sum_{j=1}^{N_{\mathcal{D}}} \sum_{i=1}^M \left[\sum_{l=1}^L a_l \lambda_x^{\text{M}} \cdot \frac{1}{T} \sum_{t=1}^T \|\hat{\mathbf{x}}_{j,i,t}^{(l)} - \mathbf{x}_{j,i,t}\|_2^2 \right. \\ &\quad \left. + \sum_{l=1}^{L-1} a_l \left(\lambda_\phi^{\text{M}} \|\hat{\phi}_{j,i}^{(l)} - \phi_{j,i}^*\|_F^2 + \lambda_\Delta^{\text{M}} \|\Delta\phi_{j,i}^{(l)}\|_F^2 \right) \right], \quad (24) \end{aligned}$$

with hyperparameters $\lambda_\phi^{\text{M}} \gg \lambda_x^{\text{M}}, \lambda_\Delta^{\text{M}} \geq 0$. Since the goal of this stage is to learn a reliable M-step update, the parameter-fit weight λ_ϕ^{M} is chosen to be dominant. The regularizer $\lambda_\Delta^{\text{M}} \|\Delta\phi\|_F^2$ discourages excessively large corrections, complementing the bounded output activation of M-Net. The per-iteration weights satisfy $a_l > 0$ and $\sum_{l=1}^L a_l = 1$, and are chosen to be increasing in l so as to emphasize the later, more refined iterations while still providing a training signal to the early ones.

Stage 3: Joint Unfolded Fine-Tuning. In the final stage, RTSNet and M-Net are trained jointly through the unfolded iterations, adapting the smoother and the learned M-step to operate in concert. The loss retains the structure of (24) from Stage 2, where $\lambda_x^{\text{M}}, \lambda_\phi^{\text{M}}$, and $\lambda_\Delta^{\text{M}}$ are replaced by $\lambda_x^{\text{E2E}}, \lambda_\phi^{\text{E2E}}$, and $\lambda_\Delta^{\text{E2E}}$, respectively, where now λ_x^{E2E} is chosen to be dominant, since the ultimate objective of EM-KalmanNet is accurate state

smoothing rather than parameter recovery. The parameter-fit term acts as an auxiliary signal guiding the adaptation.

To encourage each module to learn a robust update rule from the statistics it receives, rather than relying on the gradient flow across iterations, the gradients are detached between consecutive EM-KalmanNet iterations during training. Thus, at each iteration l , RTSNet and M-Net are trained to optimally process their inputs given the output of iteration $l - 1$ treated as fixed, effectively training each iteration as a self-contained estimation step. Beyond stabilizing training, this design has an important practical benefit: since each iteration is trained to improve upon an arbitrary incoming estimate, the trained modules remain valid when the number of iterations applied at inference deviates from L , allowing the iteration budget to be traded off against latency at deployment.

Algorithm 2: EM-KalmanNet Training

Input : Dataset $\mathcal{D}$; number of unfolded iterations L
Output : Trained parameters θ_E and θ_M
Stage 1: Pre-Train RTSNet
1 Train θ_E on $\mathcal{D}$ using $\mathcal{L}_{\mathcal{D}}^{\text{RTSNet}}$, first with $\phi_{j,i} = \phi_{j,i}^*$,
then with perturbed parameters $\phi_{j,i} = \phi_{j,i}^* + \delta\phi_{j,i}$;
Stage 2: Train M-Net with Frozen RTSNet
2 Freeze θ_E ;
3 **for** each mini-batch **do**
4 Run Algorithm 1 with mismatched initialization $\phi^{(0)}$;
5 Update θ_M using (24);
Stage 3: Joint Unfolded Fine-Tuning
6 Unfreeze θ_E ;
7 **for** each mini-batch **do**
8 Run Algorithm 1, detaching gradients between iterations;
9 Jointly update θ_E and θ_M using $\mathcal{L}_{\mathcal{D}}^{\text{E2E}}$;
10 **return** θ_E, θ_M

C. Discussion

Tackling the Formulated Problem: EM-KalmanNet addresses Challenges C.1–C.5 through dedicated components: the RTSNet-based E-step handles mismatched dynamics (C.1) and unknown, non-Gaussian noise (C.2); the parameter-aware modulation combined with the residual M-Net update tracks block-wise variations (C.3); the unfolded structure with a small fixed L ensures bounded latency (C.4); and the bounded, correction-based parameter update, together with the warm-starting of $\phi^{(0)}$ across blocks, enables reliable adaptation from short blocks (C.5), as a local refinement is learnable from far fewer samples than a full re-estimation.

Relation to Classical EM-KF: Several special cases clarify the connection between EM-KalmanNet and classical EM-KF. If RTSNet is replaced by the exact RTS smoother and M-Net by the analytical EM M-step in (12) or (13), the unfolded architecture reduces to EM-KF truncated after L iterations. By Remark 1, this analytical M-step is itself approximately realizable within the learned architecture, up to the omitted posterior-covariance terms. Conversely, setting $\Delta\phi^{(l)} = \mathbf{0}$ reduces EM-KalmanNet to RTSNet operating with a fixed model parameter. EM-KalmanNet can thus be viewed as an EM-inspired unfolded architecture

that bridges model-based EM-KF and supervised end-to-end tracking, trading the asymptotic convergence guarantees of EM (which hold only under linear-Gaussian assumptions) for learned, fixed-complexity adaptation applicable to the general model (1).

Computational Complexity: For a block of T measurements, EM-KalmanNet performs L forward-backward passes of RTSNet and $L - 1$ parameter updates by M-Net. Each RTSNet pass requires $\mathcal{O}(Tc_E)$ operations, where c_E denotes the per-time-step computational cost of RTSNet, including the learned gain networks and the available model functions. Each M-Net update requires $\mathcal{O}(Tc_S + c_M)$ operations, where c_S denotes the per-time-step cost of accumulating the required summary statistics and c_M is the cost of evaluating M-Net. Consequently, the overall per-block complexity is of order $\mathcal{O}(LTc_E + (L - 1)(Tc_S + c_M))$. For fixed network architectures and state and observation dimensions, the complexity scales linearly with both the block length T and the unfolded depth L . Since L is fixed at design time, the computational budget is known in advance, facilitating predictable low-latency inference as required by Challenge C.4.

Future Research: Several directions remain open for future study. First, the current framework can be extended to simultaneous adaptation of multiple model components, such as the dynamics and observation model, as well as to time-varying noise statistics. Second, online or streaming variants may be developed by applying the learned update mechanism over sliding windows rather than fixed blocks. Third, integration with hypernetworks [35] may allow the adaptation modules themselves to change according to the operating regime, further improving generalization across different forms of time variation and model mismatch. These aspects are left for future study.

IV. EXPERIMENTAL STUDY

We next evaluate EM-KalmanNet in several SS tracking scenarios¹. We first describe the common experimental setup (Subsection IV-A). Then, we evaluate EM-KalmanNet on controlled synthetic SS models with an unknown time-varying state-transition matrix F (Subsection IV-B) and with an unknown time-varying observation matrix H (Subsection IV-C), followed by a nonlinear Lorenz attractor model (Subsection IV-D) and an acoustic source-tracking application (Subsection IV-E).

Each experiment isolates one or more of Challenges C.1–C.5: nonlinear and chaotic dynamics test robustness to model approximation (C.1); non-Gaussian and correlated observation noise test robustness to distributional mismatch (C.2); block-wise varying F or H test adaptation to unknown time variation (C.3); reported inference latencies test computational efficiency (C.4); and a dedicated sweep over the block length T tests operation under rapidly varying, sparsely observed conditions (C.5).

A. Experimental Setup

Compared Algorithms: We compare EM-KalmanNet with several model-based and data-driven benchmarks. The model-based benchmarks include the RTS smoother [7] with full knowledge of the SS model at each block (*RTS Full*), the RTS

¹Our source code and all hyperparameters can be found at https://github.com/oricoohen/EMKF_NET.git

smoother that uses a single fixed, mismatched parameter for all blocks (*RTS False*), and the model-based EM-KF [18], initialized with the same mismatched parameter as EM-KalmanNet. As data-driven benchmarks, we evaluate RTSNet [28] operating with the same fixed mismatched parameter (*RTSNet False*), and a bidirectional GRU (BGRU) [38] smoother as a purely data-driven sequence-to-sequence benchmark. In the synthetic state-transition adaptation experiment (Subsection IV-B), we also compare with our preliminary M-step implementation (*M-RTSNETP*) [1] to isolate the gains of the proposed architecture and training procedure.

Within each experiment, all data-driven methods are trained using the same training dataset of $N_D = 400$ trajectories, each consisting of M blocks of length T , where M and T are experiment-specific and stated in each subsection. EM-KalmanNet is trained using the unfolded procedure described in Subsection III-B, with the parameter-aware RTSNet and M-Net being shared across iterations. For a controlled comparison, both adaptive methods use a small, fixed iterative budget of 3 by default. This fixed computational budget is consistent with the latency requirement of Challenge C.4; Subsection IV-D additionally reports an EM-KF variant with a larger budget for reference.

Performance Metrics and Noise Levels: Performance is evaluated using the state-estimation mean squared error (MSE) in dB, averaged over the test trajectories and time indices. In all experiments except the observation-noise-distribution experiment, the process- and observation-noise covariance matrices are parameterized as $\mathbf{Q} = q^2 \mathbf{I}_m$ and $\mathbf{R} = r^2 \mathbf{I}_n$, respectively, where q^2 and r^2 control the corresponding noise levels. Depending on the experiment, either q^2 is held fixed while r^2 is varied, or the ratio q^2/r^2 is kept fixed while both are scaled jointly. Accordingly, when results are reported versus $1/r^2$ in dB, larger values correspond to cleaner observations.

B. Synthetic State-Transition Adaptation (Case 1)

1) *Setup:* In all three experiments, each test trajectory consists of three blocks of length $T = 30$. The reference dynamics is set by the nominal state-transition matrix,

$$\mathbf{F}_0 = \begin{bmatrix} 0.83 & 0.20 \\ 0.20 & 0.83 \end{bmatrix}. \quad (25)$$

The state-transition matrix of the first block is obtained by rotating $\mathbf{F}_0$ by 0.2 radians. Each subsequent block applies an additional 0.2-radian rotation to that of the preceding block. In this experiment, we fix $q^2 = 0.01$ and vary r^2 . Thus, the reported $1/r^2$ axis in dB directly reflects changes in the observation signal-to-noise ratio (SNR). The three experiments differ only in the observation model: (i) a linear observation model

$$\mathbf{H} = \begin{bmatrix} 1 & 1 \\ 0.25 & 1 \end{bmatrix} \quad (26)$$

with Gaussian noise; (ii) the same $\mathbf{H}$ with exponentially distributed noise; and (iii) a nonlinear observation model

$$h(\mathbf{x}) = \mathbf{H}\mathbf{x} + 0.3h_0(\mathbf{x}), \quad h_0(\mathbf{x}) = \begin{bmatrix} \sqrt{x_1^2 + x_2^2} \\ \text{atan2}(x_2, x_1) \end{bmatrix}. \quad (27)$$

The training trajectories follow the same three-block structure as the test trajectories. The first block uses $\mathbf{F}_0$ rotated by angle

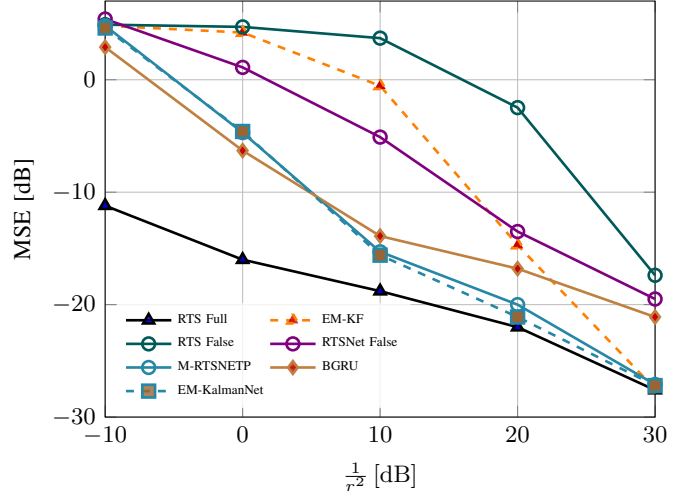


Fig. 2: MSE vs. SNR, linear Gaussian SS model, Case 1.

sampled from $\mathcal{U}[-1, 1]$ radians. Each subsequent state-transition matrix is obtained by rotating using a newly sampled rotation angle from the same distribution. This exposes the data-driven methods to a range of block-wise model variations during training.

2) *Linear Gaussian Setting:* Fig. 2 reports the state-estimation MSE. As expected, RTS Full provides the optimal benchmark, since it operates with the true block-specific state-transition matrices under the linear Gaussian assumptions.

At low SNR, BGRU achieves the best performance among the non-oracle methods. In this regime, an MMSE optimal estimator relies more on the dynamical prior than on highly noisy observations, and BGRU's bidirectional recurrent architecture learns such temporal dependencies directly from the training data, without an explicit SS model. As SNR increases, EM-KalmanNet and M-RTSNETP increasingly outperform BGRU, since their model-based structure exploits the observation-state relation explicitly, which becomes more valuable as observations grow more informative. EM-KalmanNet closely tracks M-RTSNETP over the considered SNR range, with a small advantage at moderate-to-high SNR. The relatively narrow gap between the two methods is expected, since the analytical M-step employed by M-RTSNETP is well matched to the linear Gaussian setting, leaving limited room for the learned M-Net to improve upon it.

3) *Linear Non-Gaussian Setting:* Fig. 3 reports the results for the linear model with exponentially distributed observation noise. Because the model-based smoothers' Gaussian assumption is now violated, RTS Full is no longer guaranteed to be MSE-optimal, and EM-KalmanNet closely tracks it throughout the tested range while consistently outperforming every other non-oracle method. The advantage over M-RTSNETP is substantially larger here than in the Gaussian setting, which can be explained by the fact that M-RTSNETP's analytical M-step is derived under linear Gaussian assumptions, whereas M-Net of the EM-KalmanNet is free to adapt its parameter-update rule to the non-Gaussian statistics seen during training. Similarly, EM-KF degrades sharply when the observation noise is dominant, but recovers at high SNR, nearly matching the best-performing methods. As in the Gaussian experiment, BGRU improves more gradually as the SNR increases, widening its gap to the model-guided methods.

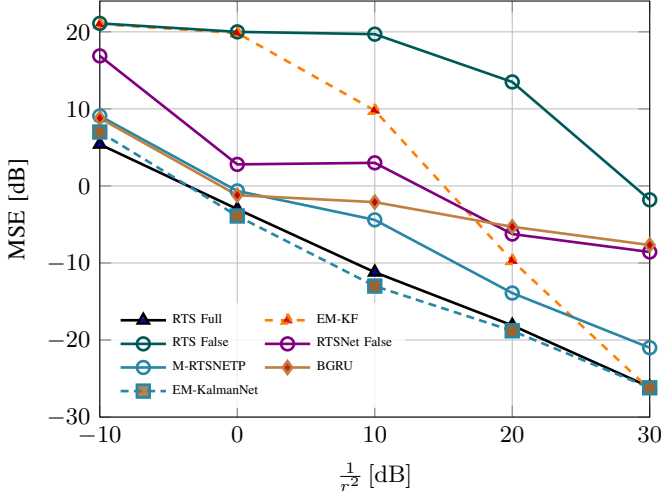


Fig. 3: MSE vs. SNR, linear non-Gaussian SS model, Case 1.

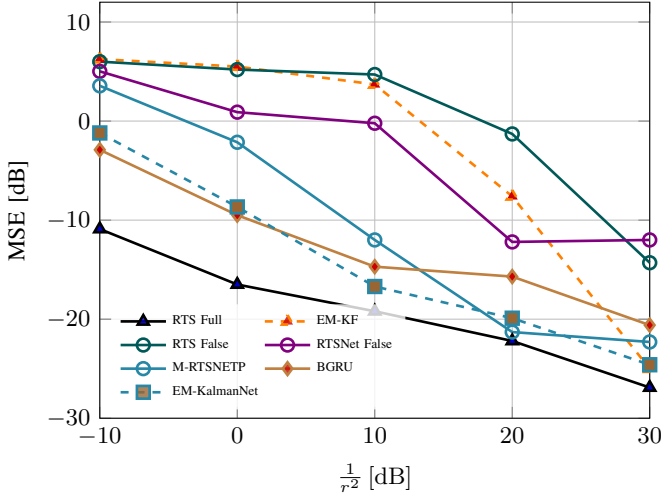


Fig. 4: MSE vs. SNR, nonlinear observation model, Case 1.

These results demonstrate that EM-KalmanNet remains effective under non-Gaussian observation noise, addressing Challenge C.2.

4) *Nonlinear Observation Setting*: Fig. 4 reports the results for the nonlinear observation model, where RTS Full remains the strongest benchmark due to the accurate local linearization of $h(\cdot)$. EM-KalmanNet is nonetheless among the strongest non-oracle methods across the tested SNR range: BGRU performs slightly better at the two lowest SNR levels, EM-KalmanNet leads clearly at moderate-to-high SNR, and at the highest SNR, EM-KF closely matches EM-KalmanNet while M-RTSNETP becomes competitive as well. Overall, EM-KalmanNet maintains strong performance under nonlinear observation models, addressing Challenge C.1.

Table I reports inference latency per sequence for the nonlinear observation experiment, addressing Challenge C.4. All timings are evaluated on an NVIDIA GeForce RTX 4070 SUPER GPU platform using PyTorch, hence the notably lower latency of the BGRU, which is highly accelerated by PyTorch. Although EM-KF is restricted to only 3 EM iterations, EM-KalmanNet remains considerably faster, reducing the latency

TABLE I: Inference latency, nonlinear synthetic experiment, Case 1.

Method	Latency [ms/sequence]
RTS	190
RTSNet	180
EM-KF, 3 iterations	600
M-RTSNETP	512
BGRU	0.23
EM-KalmanNet	335

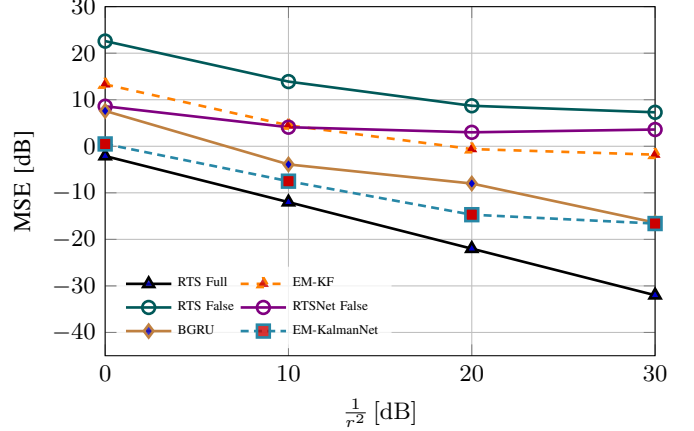


Fig. 5: MSE vs. SNR, linear Gaussian SS model, Case 2.

from 600 to 335 [ms] ($\approx 44.2\%$), and relative to M-RTSNETP from 512 to 335 [ms] ($\approx 34.6\%$), while achieving comparable or better estimation accuracy across the tested SNR range (Figs. 2–4). This gap between EM-KalmanNet and the iterative model-based baselines widens further in the more demanding settings considered later in this section.

C. Synthetic Observation-Matrix Adaptation (Case 2)

1) *Setup*: The experimental setup follows that of Subsection IV-B, except that the unknown block-wise varying parameter is now $\mathbf{H}$, while the state-transition matrix $\mathbf{F}_0$ remains fixed and known. The nominal observation matrix $\mathbf{H}_0$ is as in (26). The observation matrix of the first block is obtained by rotating $\mathbf{H}_0$ by 0.2 radians, and each subsequent block applies an additional rotation of 0.2 radians. We set $q^2 = r^2$, such that the process noise is non-negligible relative to the measurement noise throughout; given the substantial performance degradation observed for all methods at $1/r^2 = -10$ dB, we report here $1/r^2 \in \{0, 10, 20, 30\}$ dB.

2) *Linear Gaussian Setting*: Fig. 5 shows that, with RTS Full as the oracle benchmark, EM-KalmanNet generally achieves the strongest non-oracle MSE performance, and substantially outperforms both RTS False and RTSNet False, demonstrating the benefit of adapting the observation matrix across blocks. It also consistently improves over EM-KF, while BGRU is competitive only at the lowest observation-noise variance. These results demonstrate that the learned M-Net can effectively adapt the unknown observation matrix $\mathbf{H}$, not merely $\mathbf{F}$, and that this holds even when the process noise remains comparable to the observation noise throughout the tested range.

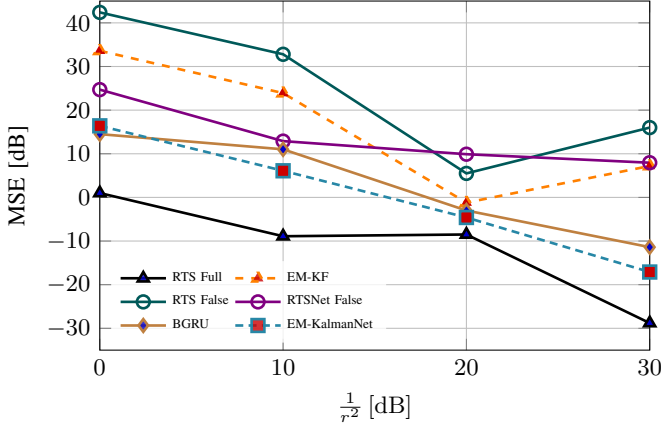


Fig. 6: MSE vs. SNR, linear non-Gaussian SS model, Case 2.

3) *Linear Non-Gaussian Setting*: Fig. 6 reports the results under exponential observation noise. RTS Full remains the strongest empirical benchmark, although it is no longer guaranteed to be MSE-optimal under the non-Gaussian noise distribution. EM-KalmanNet achieves the strongest non-oracle MSE performance over most of the tested observation-noise levels, with BGRU performing slightly better only at the lowest $1/r^2$. Together with the preceding Gaussian results, these results show that M-Net can effectively adapt the unknown block-varying observation matrix $\mathbf{H}$ under non-Gaussian observation noise as well, thereby addressing Challenge C.2.

D. Lorenz Attractor

The Lorenz attractor experiments [39] revisit each of the challenges introduced in Section II: The chaotic Lorenz dynamics constitute a highly nonlinear, hard-to-approximate model (Challenge C.1). Building on this model, we first evaluate adaptation to substantial changes in the observation matrix across datasets (Challenge C.3); we report the inference latency (Challenge C.4); we then evaluate robustness to different observation-noise distributions (Challenge C.2); and finally, we evaluate operation across a range of block lengths (Challenge C.5).

1) *Setup*: The state evolves according to the discretized Lorenz attractor, a three-dimensional chaotic solution of the Lorenz ordinary differential equations, leading to the standard Lorenz-63 state evolution (see [27, Sec. IV-D]). The observation model is linear in the state and is given by $\mathbf{y}_t = \mathbf{H}_i \mathbf{x}_t + \mathbf{v}_t$, where $\mathbf{H}_i$ is the observation matrix used in the i th block. Throughout this subsection, the unknown block-wise time-varying parameter is $\mathbf{H}$, while the Lorenz state-transition function is fixed and known.

All compared methods are initialized with the nominal matrix $\mathbf{H}_0 = \mathbf{R}_{\text{rot}}(\alpha_0)$, where $\mathbf{R}_{\text{rot}}(\cdot)$ is the three-dimensional rotation matrix and $\alpha_0 = 1^\circ$. RTS False and RTSNet False retain $\mathbf{H}_0$ throughout all datasets; the adaptive methods (EM-KF, EM-KalmanNet) are subsequently warm-started using their own estimate from the preceding dataset.

2) *Observation-Matrix Adaptation*: Here, the test sequence comprises five consecutive datasets, each of length $T = 30$, with

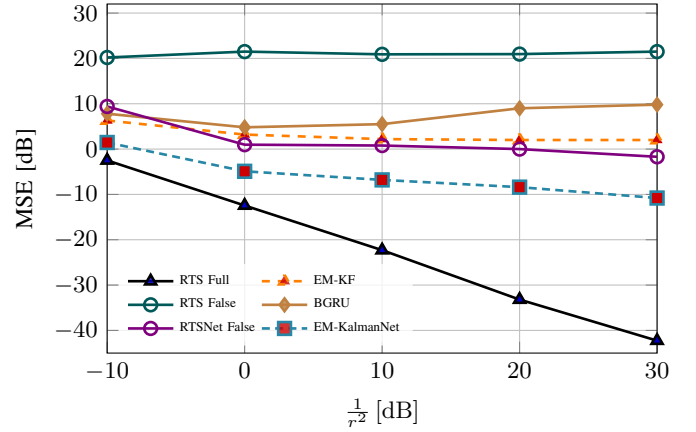


Fig. 7: MSE vs. SNR, Lorenz attractor, Case 2.

observation matrices generated by a fixed cumulative rotation,

$$\mathbf{H}_0^{\text{test}} = \mathbf{H}_0, \quad (28)$$

$$\mathbf{H}_i^{\text{test}} = \mathbf{R}_{\text{rot}}(0.3) \mathbf{H}_{i-1}^{\text{test}}, \quad i = 1, \dots, 5, \quad (29)$$

applying the same 0.3-radian angle about all three coordinate axes. Training trajectories share this five-dataset structure, but each rotation angle is instead drawn independently with

$$\mathbf{H}_0^{(j)} = \mathbf{I}_3, \quad (30)$$

$$\mathbf{H}_i^{(j)} = \mathbf{R}_{\text{rot}}(\psi_i^{(j)}) \mathbf{H}_{i-1}^{(j)}, \quad i = 1, \dots, 5, \quad (31)$$

where the rotation angle is independently sampled for each dataset and trajectory from $\mathcal{U}[-0.3, 0.3]$ radians for the three coordinate axes. The test setting is particularly challenging: the true rotation is applied cumulatively at the boundary of the training range, so the test trajectories exhibit a systematic drift only sparsely represented during training. Strong performance here indicates that EM-KalmanNet learns a transferable adaptation strategy rather than memorizing training trajectories.

Fig. 7 shows this mismatch strongly degrades BGRU, whose fully data-driven operation depends heavily on the similarity between the training and test trajectories. EM-KalmanNet is more robust, as it combines learned inference with explicit SS structure and block-wise adaptation of $\mathbf{H}$, and therefore outperforms all non-oracle methods. It also improves over EM-KF, which struggles to recover the large accumulated changes within the limited number of iterations. Due to the substantial cumulative variation in the observation model, none of the adaptive methods approaches the oracle RTS Full, which receives the true $\mathbf{H}_i$ in every dataset. Nevertheless, EM-KalmanNet is the strongest and most consistent among the non-oracle methods.

Table II reports the latency per complete five-block sequence for this nonlinear, chaotic setting. Despite its 3-iteration budget, EM-KalmanNet reduces the latency relative to EM-KF from 868 to 242 [ms] ($\approx 72.1\%$); RTSNet False and BGRU are faster still, but at substantially inferior estimation accuracy.

3) *Robustness Across Observation-Noise Distributions*: We next repeat the preceding Lorenz experiment under six different observation-noise distributions, keeping the remaining setup unchanged, to evaluate robustness beyond the Gaussian noise assumption (Challenge C.2). Alongside standard Gaussian noise,

Method	Latency [ms/sequence]
RTS Full	280
EM-KF, 3 iterations	868
BGRU	0.4
RTSNet False	83
EM-KalmanNet	242

TABLE II: Inference latency, Lorenz attractor, $T = 30$.

Method	Gaussian	Correlated Gaussian	Laplace	Exponential	Student- t	Contaminated
RTS Full	-12.43 ± 2.50	-12.90 ± 2.47	-12.30 ± 2.45	-12.36 ± 2.48	-10.20 ± 2.80	-11.50 ± 2.50
RTSNet False	-0.97 ± 4.70	1.08 ± 5.10	-0.90 ± 5.00	-1.80 ± 4.37	-1.20 ± 4.39	-1.20 ± 4.50
EM-KF	2.50 ± 4.60	7.59 ± 3.66	2.75 ± 4.39	2.71 ± 4.50	2.88 ± 4.33	3.47 ± 4.47
BGRU	4.80 ± 8.70	7.96 ± 8.57	6.30 ± 9.17	5.50 ± 8.85	6.70 ± 8.98	3.28 ± 7.50
EM-KalmanNet	-4.90 ± 4.60	-3.36 ± 5.48	-6.30 ± 4.10	-4.20 ± 4.82	-3.86 ± 4.34	-6.20 ± 4.00

TABLE III: State-estimation MSE (mean $\pm$ std) under different observation-noise distributions, Lorenz attractor.

we consider a correlated Gaussian setting. For the correlated Gaussian setting, the observation-noise covariance is

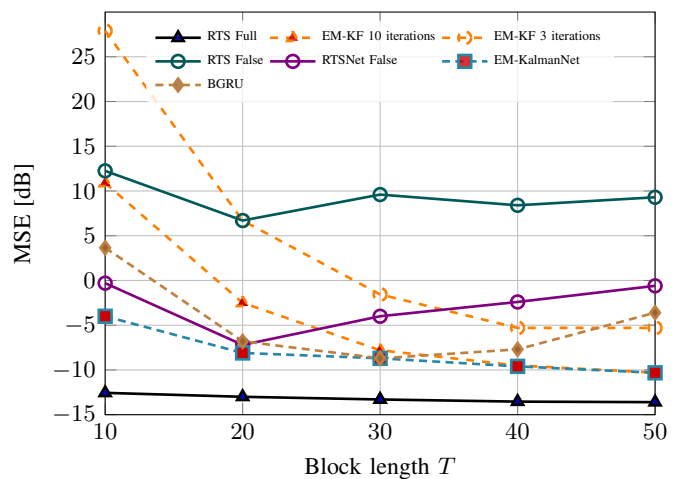
$$\mathbf{R} = r^2 \begin{bmatrix} 1 & 0.6 & 0.6 \\ 0.6 & 1 & 0.6 \\ 0.6 & 0.6 & 1 \end{bmatrix},$$

such that we have equal per-channel variance r^2 and pairwise correlation $\rho = 0.6$. In addition, we have symmetric, heavy-tailed Laplace noise; Student- t noise with $\nu = 4$ degrees of freedom, producing frequent large-amplitude deviations; exponential noise, centered to zero mean while retaining its asymmetric, skewed shape; and contaminated Gaussian noise, modeling a predominantly Gaussian process with rare high-amplitude outliers ($\epsilon = 0.05$, $\kappa = 3$). These settings jointly probe cross-channel correlation, tail heaviness, asymmetry, and impulsiveness.

Table III shows that EM-KalmanNet achieves the lowest MSE among the non-oracle methods across all tested noise settings, with a particularly clear margin under the heavy-tailed Laplace and contaminated Gaussian distributions. The correlated Gaussian case is hardest for all methods, as the shared cross-channel noise component reduces the independent information available in the observations, yet EM-KalmanNet's advantage persists. Its standard deviations are also consistently smaller than BGRU's, indicating more reliable performance across test realizations rather than a strong dependence on specific temporal patterns seen in training. Overall, these results demonstrate the robustness of EM-KalmanNet to diverse noise statistics, addressing Challenge C.2.

4) *Effect of the Block Length*: We next vary T to evaluate adaptation to the changing observation model when only a limited number of observations is available, addressing Challenge C.5. We set $r^2 = 1$ and $q^2 = 0.01$, and consider three consecutive blocks with a cumulative rotation of 0.2 radians between them. We additionally report EM-KF with 10 iterations to assess whether a larger iterative budget can compensate for short blocks.

Fig. 8 reports the MSE versus different block lengths. RTS Full provides the strongest performance, whereas RTS False remains severely degraded by the fixed model mismatch. EM-

Fig. 8: MSE vs. T , Lorenz attractor, $r^2 = 1$, $q^2 = 0.01$.

KalmanNet maintains the best non-oracle performance across all tested T , with its advantage most pronounced for $T < 30$. Even the 10-iteration EM-KF variant cannot close this gap at short T , despite its added latency; EM-KF only approaches EM-KalmanNet as T grows and more observations become available. BGRU is competitive for small T , where short-range temporal patterns are easier to capture from the training data, but its performance degrades as T increases. RTSNet False is similarly competitive for short blocks but, lacking explicit adaptation of $\mathbf{H}$, benefits less from longer blocks, leading to a growing gap to EM-KalmanNet. These results demonstrate that EM-KalmanNet can adapt from a limited number of observations, addressing C.5.

5) *Stability Over an Extended Horizon*: We next evaluate repeated adaptation over ten consecutive datasets of length $T = 30$, with $\mathbf{H}$ fixed within each dataset and cumulatively rotated by $\psi = 0.1$ radians between datasets, using a harder noise level $r^2 = 10$, $q^2 = 0.1$. Unlike the five-dataset experiment above, which considers larger changes over fewer datasets, this setting applies smaller but more frequent changes over a longer horizon. The goal is to assess whether successive parameter updates remain stable and whether estimation errors accumulate over time.

Table IV shows that EM-KalmanNet achieves the strongest performance among the non-oracle methods. These results indicate that the repeated M-Net updates remain effective over the extended sequence without substantial error accumulation. Moreover, EM-KalmanNet exhibits a relatively low standard deviation of 2.6 dB, close to that of the oracle RTS Full benchmark. Fig. 9 further illustrates how these numerical differences translate into the reconstructed state trajectories. EM-KalmanNet closely preserves the characteristic geometry of the Lorenz attractor and follows the ground-truth trajectory more faithfully than the other non-oracle methods.

E. Acoustic Tracking

We finally consider a real-world-inspired acoustic tracking scenario, in which a moving sound source with varying motion dynamics is localized from measurements collected by a microphone array. This experiment represents a practical setting where the source motion is not accurately known a priori and the

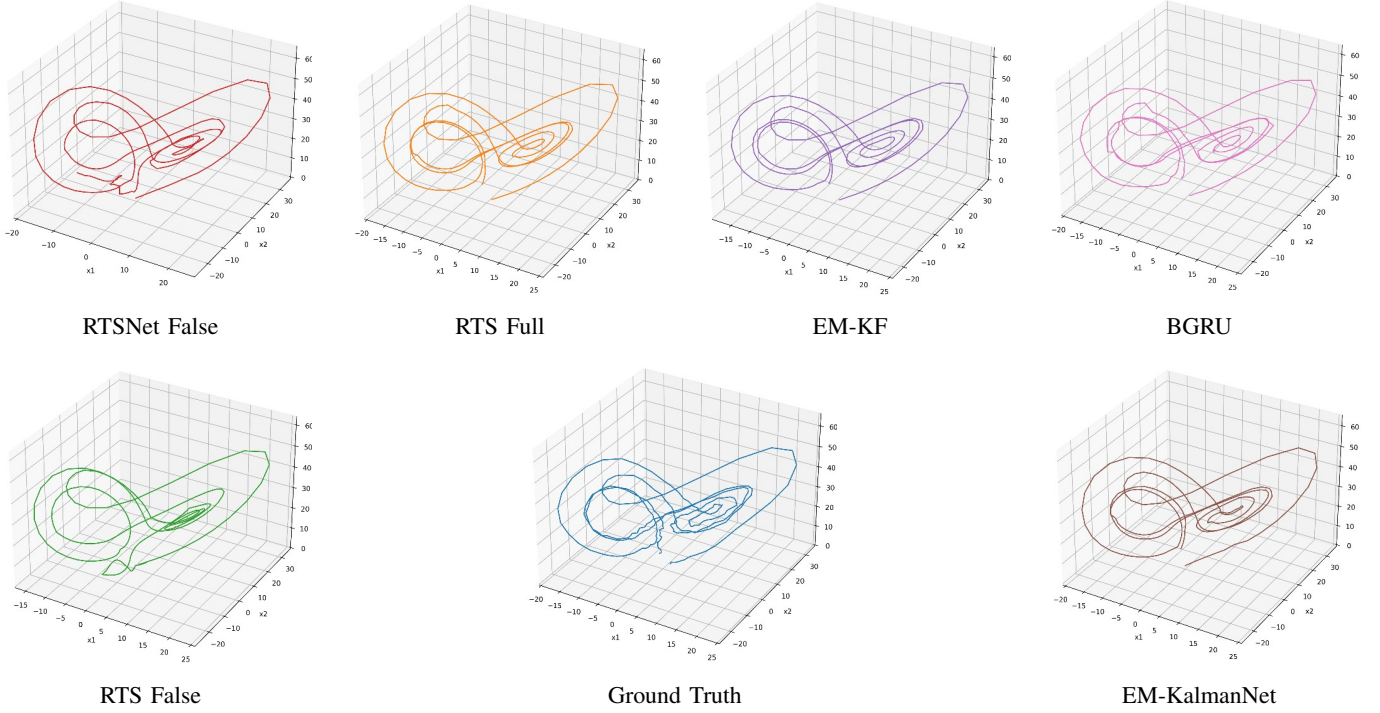


Fig. 9: Representative state-estimation trajectories for the Lorenz attractor experiment with ten consecutive datasets, and a cumulative observation-model rotation of $\psi = 0.1$ radians between successive datasets.

RTS Full	RTS False	RTSNet False	EM-KF	BGRU	EM-KalmanNet
-3.3 ± 2.3	14.8 ± 7.4	12.1 ± 7.6	4.4 ± 4	1.7 ± 3.48	-0.86 ± 2.6

TABLE IV: State-estimation MSE (mean $\pm$ standard deviation) for the Lorenz attractor experiment with ten consecutive datasets, $T = 30$, $r^2 = 10$, and an observation-model rotation of $\psi = 0.1$ radians between successive datasets.

observations are nonlinear, allowing us to evaluate the applicability of EM-KalmanNet beyond the preceding synthetic setups [40].

1) *Setup*: The hidden state consists of the two-dimensional source position and velocity,

$$\mathbf{x}_t = [p_{x,t} \ p_{y,t} \ v_{x,t} \ v_{y,t}]^\top. \quad (32)$$

The state evolves according to $\mathbf{x}_t = \mathbf{F}(\psi)\mathbf{x}_{t-1} + \mathbf{w}_t$, where

$$\mathbf{F}(\psi) = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & \cos \psi & -\sin \psi \\ 0 & 0 & \sin \psi & \cos \psi \end{bmatrix}, \quad \Delta t = 0.1. \quad (33)$$

Thus, the position follows constant-velocity kinematics, while the velocity vector is rotated by an angle ψ . The structure of $\mathbf{F}(\psi)$ is known, whereas ψ , and consequently the block-specific state-transition matrix, is unknown.

The observations are time-difference-of-arrival measurements relative to a reference microphone,

$$y_t^{(i)} = \frac{\|\mathbf{p}_t - \mathbf{s}_i\|_2 - \|\mathbf{p}_t - \mathbf{s}_{\text{ref}}\|_2}{c} + v_t^{(i)}, \quad (34)$$

where $\mathbf{p}_t = [p_{x,t}, p_{y,t}]^\top$ and c denotes the propagation velocity.

Hence, the observations are nonlinear functions of the source position. The process- and observation-noise levels are set to $q^2 = 0.01$ and $r^2 = 1$, respectively.

Each trajectory comprises six temporally consecutive blocks. The final state of each block initializes the following block, while the motion parameter ψ remains fixed within a block and changes between blocks. During training, ψ is independently sampled for each block and trajectory according to $\psi \sim \mathcal{U}[-0.12, 0.12]$. During testing, the six blocks use the fixed sequence $\psi = [0.1, 0.08, -0.1, -0.08, 0.06, 0.1]$. The mismatched baselines assume $\psi = 0$. As in the previous experiments, EM-KalmanNet is initialized with $\mathbf{F}(0)$ at each block and estimates the block-specific transition matrix without warm-starting from the preceding block.

2) *Results*: Fig. 10 illustrates the acoustic tracking experiment, while Table V reports the overall state-estimation MSE. As expected, ERTS with the true block-specific transition matrix provides the strongest oracle benchmark. In contrast, ERTS and RTSNet operating with the fixed mismatched model $\psi = 0$ degrade when the actual source motion departs from the assumed constant-velocity dynamics. EM-KalmanNet substantially improves upon these mismatched baselines by adapting the state-transition matrix to the motion in each block. Notably, this adaptation is performed from the same mismatched initialization at the beginning of every block, demonstrating the ability of EM-KalmanNet to repeatedly recover changing motion dynamics without relying on the parameter estimate obtained in the preceding block.

V. CONCLUSION

We considered state estimation in partially known state-space models with block-wise time-varying parameters. We proposed

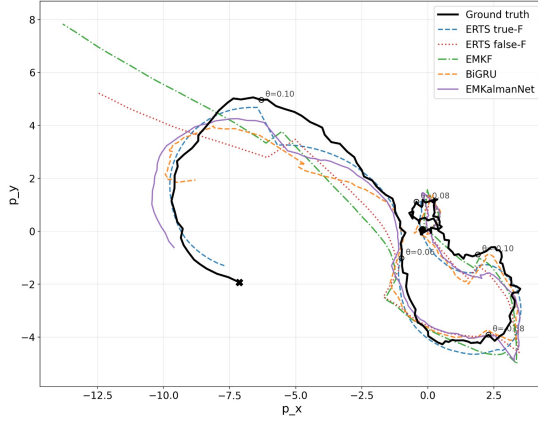


Fig. 10: Acoustic tracking experiment with three microphones.

Method	MSE [dB]
RTS True	-8.39
RTS False	-1.95
RTSNet False- $\mathbf{F}$	-4.48
BGRU	-4.73
EM-KalmanNet	-6.1

TABLE V: Overall MSE for the acoustic tracking experiment.

EM-KalmanNet, which combines an RTSNet-based learned E-step with a data-driven M-step for iterative state smoothing and parameter adaptation. The numerical study shows that EM-KalmanNet consistently improves over mismatched model-based and data-driven benchmarks across linear and nonlinear dynamics, non-Gaussian noise, and rapidly varying model conditions, while maintaining a fixed and practical inference budget.

REFERENCES

- [1] O. Cohen, X. Ni, N. Shlezinger, and T. Routtenberg, "EM-KalmanNet: AI-aided Kalman tracking in partially known time-varying state-space models," in *IEEE Sensor Array and Multichannel Signal Processing Workshop (SAM)*, 2026.
- [2] S. Särkkä and L. Svensson, *Bayesian filtering and smoothing*. Cambridge university press, 2023, vol. 17.
- [3] Z. Karami and R. Kashef, "Smart transportation planning: Data, models, and algorithms," *Transportation Engineering*, vol. 2, p. 100013, 2020.
- [4] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with applications to tracking and navigation: theory algorithms and software*. John Wiley & Sons, 2004.
- [5] J. Durbin and S. J. Koopman, *Time series analysis by state space methods*. Oxford University Press, 2012.
- [6] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [7] H. E. Rauch, F. Tung, and C. T. Striebel, "Maximum likelihood estimates of linear dynamic systems," *AIAA Journal*, vol. 3, no. 8, pp. 1445–1450, 1965.
- [8] S. F. Schmidt, "The Kalman filter-its recognition and development for aerospace applications," *Journal of Guidance and Control*, vol. 4, no. 1, pp. 4–7, 1981.
- [9] S. J. Julier and J. K. Uhlmann, "Unscented filtering and nonlinear estimation," *Proc. IEEE*, vol. 92, no. 3, pp. 401–422, 2004.
- [10] I. Arasaratnam and S. Haykin, "Cubature Kalman filters," *IEEE Trans. Autom. Control*, vol. 54, pp. 1254–1269, 2009.
- [11] R. Mehra, "On the identification of variances and adaptive Kalman filtering," *IEEE Trans. Autom. Control*, vol. 15, no. 2, pp. 175–184, 1970.
- [12] H. A. Blom and Y. Bar-Shalom, "The interacting multiple model algorithm for systems with markovian switching coefficients," *IEEE Trans. Autom. Control*, vol. 33, no. 8, pp. 780–783, 1988.
- [13] E. A. Wan and A. T. Nelson, "Dual extended Kalman filter methods," *Kalman filtering and neural networks*, pp. 123–173, 2001.
- [14] R. Togneri and L. Deng, "Joint state and parameter estimation for a target-directed nonlinear dynamic system model," *IEEE Trans. Signal Process.*, vol. 51, no. 12, pp. 3061–3070, 2003.
- [15] L. Zhang, D. Sidoti, A. Bienkowski, K. R. Pattipati, Y. Bar-Shalom, and D. L. Kleinman, "On the identification of noise covariances and adaptive Kalman filtering: A new look at a 50 year-old problem," *IEEE Access*, vol. 8, pp. 59 362–59 388, 2020.
- [16] T. K. Moon, "The expectation-maximization algorithm," *IEEE Signal Process. Mag.*, vol. 13, no. 6, pp. 47–60, 1996.
- [17] S. Gannot and A. Yeredor, "The Kalman filter," *Springer Handbook of Speech Processing*, pp. 135–160, 2008.
- [18] R. H. Shumway and D. S. Stoffer, "An approach to time series smoothing and forecasting using the EM algorithm," *Journal of Time Series Analysis*, vol. 3, pp. 253–264, 1982.
- [19] N. Shlezinger *et al.*, "Artificial intelligence-aided Kalman filters: AI-augmented designs for Kalman-type algorithms," *IEEE Signal Process. Mag.*, vol. 42, no. 3, pp. 52–76, 2025.
- [20] T. Haarnoja, A. Ajay, S. Levine, and P. Abbeel, "Backprop KF: Learning discriminative deterministic state estimators," *Advances in Neural Information Processing Systems*, vol. 29, 2016.
- [21] P. Becker, H. Pandya, G. Gebhardt, C. Zhao, C. J. Taylor, and G. Neumann, "Recurrent Kalman networks: Factorized inference in high-dimensional deep feature spaces," in *International Conference on Machine Learning*, 2019, pp. 544–552.
- [22] A. Ghosh, A. Honoré, and S. Chatterjee, "DANSE: Data-driven non-linear state estimation of model-free process in unsupervised learning setup," *IEEE Trans. Signal Process.*, vol. 72, pp. 1824–1838, 2024.
- [23] V. G. Satorras, Z. Akata, and M. Welling, "Combining generative and discriminative models for hybrid inference," *Advances in Neural Information Processing Systems*, 2019.
- [24] A. Klushyn *et al.*, "Latent matters: Learning deep state-space models," *Advances in Neural Information Processing Systems*, 2021.
- [25] N. Shlezinger, J. Whang, Y. C. Eldar, and A. G. Dimakis, "Model-based deep learning," *Proc. IEEE*, vol. 111, no. 5, pp. 465–499, 2023.
- [26] L. Xu and R. Niu, "EKFNet: Learning system noise covariance parameters for nonlinear tracking," *IEEE Trans. Signal Process.*, vol. 72, pp. 3139–3152, 2024.
- [27] G. Revach *et al.*, "KalmanNet: Neural network aided Kalman filtering for partially known dynamics," *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.
- [28] —, "RTSNet: Learning to smooth in partially known state-space models," *IEEE Trans. Signal Process.*, vol. 71, pp. 4441–4456, 2023.
- [29] G. Choi *et al.*, "Split-KalmanNet: A robust model-based deep learning approach for state estimation," *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12 326–12 331, 2023.
- [30] I. Buchnik *et al.*, "Latent-KalmanNet: Learned Kalman filtering for tracking from high-dimensional signals," *IEEE Trans. Signal Process.*, vol. 72, pp. 352–367, 2023.
- [31] J. Wang, X. Geng, and J. Xu, "Nonlinear Kalman filtering based on self-attention mechanism and lattice trajectory piecewise linear approximation," *arXiv preprint arXiv:2404.03915*, 2024.
- [32] G. Revach, N. Shlezinger, T. Locher, X. Ni, R. J. van Sloun, and Y. C. Eldar, "Unsupervised learned Kalman filtering," in *European Signal Processing Conference (EUSIPCO)*. IEEE, 2022, pp. 1571–1575.
- [33] W. Zhang, X. Ni, N. Shlezinger, and Z. Wang, "Change-aware self-adaptive AI-aided Kalman filters with neural change point detection," *arXiv preprint arXiv:2026.13387*, 2026.
- [34] S. Chen *et al.*, "MAML-KalmanNet: A neural network-assisted Kalman filter based on model-agnostic meta-learning," *IEEE Trans. Signal Process.*, vol. 73, pp. 988–1003, 2025.
- [35] X. Ni, G. Revach, and N. Shlezinger, "Adaptive KalmanNet: Data-driven Kalman filter with fast adaptation," in *Proc. IEEE ICASSP*, 2024.
- [36] N. Shlezinger and T. Routtenberg, "Discriminative and generative learning for linear estimation of random signals," *IEEE Signal Process. Mag.*, vol. 40, no. 6, pp. 75–82, 2023.
- [37] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 39, no. 1, pp. 1–38, 1977.
- [38] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [39] E. N. Lorenz, "Deterministic nonperiodic flow," *Journal of the Atmospheric Sciences*, vol. 20, no. 2, pp. 130–141, 1963.
- [40] S. Gannot and T. G. Dvorkind, "Microphone array speaker localizers using spatial-temporal information," *EURASIP Journal on Applied Signal Processing*, vol. 2006, pp. 1–17, 2006.